

HERENCIA-EXPLICADA-JAVARUBY.pdf



TuGitanoDelInfo



Programación y Diseño Orientado a Objetos



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada**



La mejor escuela de negocios en energía, sostenibilidad y medio ambiente de España.

Más información
www.eoi.es

Formamos
talento para un futuro
Sostenible



100% Empleabilidad



Modalidad: Presencial u online



**Programa de Becas,
Bonificaciones y Descuentos**



**YA TIENES UN TÍTULO,
AHORA DA EL SALTO AL
MUNDO LABORAL.**

**POTENCIA TU PERFIL
CON LAS TECNOLOGÍAS
MÁS DEMANDADAS.**



Servicio de carreras
para que encuentres
curro en 180 días.



TEORIA PDOO

09/11/2023

RELACIONES DE HERENCIA

```
public class CantanteUrbano extends Cantante
```

PADRE

```
public class Cantante {  
    public String nombreArtistico;  
  
    public Cantante(String nombreArtistico) {  
        this.nombreArtistico = nombreArtistico;  
    }  
  
    public void cantar(){  
        System.out.println("eo");  
    }  
}
```

HIJO

```
public class CantanteUrbano extends Cantante{  
    public CantanteUrbano(String nombreA){  
        super(nombreA);  
    }  
    @Override //PARA REDEFINIR METODO DEL PADRE  
    public void cantar(){
```

```

        super.cantar();
        System.out.println(nombreArtistico);
    }
}

```

EJEMPLO SEBOLLA CON CONSTRUCTORES

```

ANIMAL
public Animal (int e){
    edad = e;
}

MAMIFERO
public Mamifero (int e,String pel){
    Super (e);
    pelaje = pel;
}

PERRO
public Perro (int e,String p,String r){
    super(e,p);
    raza = r;
}

```

HERENCIA EN RUBY Animal > Mamifero > Perro

```

# frozen_string_literal: true

class Animal
  def initialize(edad)
    @edad= edad
  end

end

class Mamifero < Animal
  def initialize(edad,pelaje)
    super(edad)
    @pelaje=pelaje
  end
end

class Perro < Mamifero

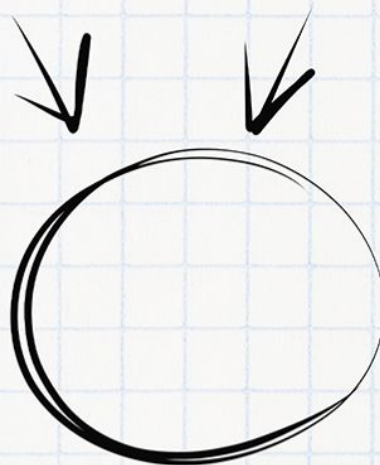
```

Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?

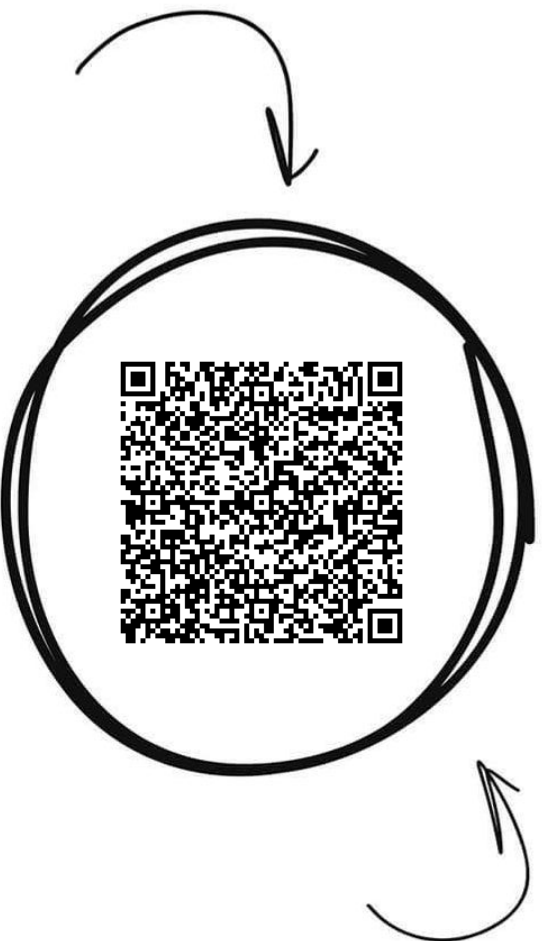


WUOLAH

Programación y Diseño Orient...



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

1 Imprime esta hoja

2 Recorta por la mitad

3 Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

4 Llévate dinero por cada descarga de los documentos descargados a través de tu QR



```
def initialize(edad, pelaje, raza)
  super(edad, pelaje) #LLAMADA AL CONSTRUCTOR DEL PADRE
  @raza= raza
end
end

p=Perro.new(8, "marron", "pastor")
puts p.inspect
```

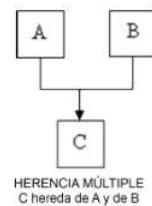
- Animal es lo que tienen en común todas las otras clases hijas
- Animal es una generalización de las otras clases
- Y las subclases hijas son especialización de Animal

Moto no es hija de Coche en todo caso ser una relación de Agregación

HERENCIA MULTIPLE:

Tener varias super clases, muchos
paes

No lo vamos a usar xd



USOS DE “SUPER” EN JAVA:

- super (nomb) ; //PARA EJECUTAR LOS MÉTODOS DEL PADRE
- super (atributo del pae) _en un método o constructor del hijo



SI EL MÉTODO DEL PADRE NO ESTÁ REDEFINIDO EN EL HIJO NO HACE FALTA USAR “SUPER” para usar el método del padre en el hijo

¡GAME OVER A LAS CLASES DE TEORÍA!



x3



x3

CONVIÉRTETE EN PROGRAMADOR
Y APRENDE PRACTICANDO

(EN SOLO 18 SEMANAS)

Dentro de un metodo de un hijo en ruby usando solamente `super` llama al padre y sus cosas

```
class CantanteUrbano
  def initialize(na)
    super # EQUIVALE A PONER super(na) en java
  end

  def cantar
    super #Equivale a super.cantar en java
    puts kfwnofa
  end
end
```

Dentro de un metodo de un hijo en ruby usando solamente `super` llama al padre y sus cosas

```
public void dimeDiscos(){
    int i = discografia.size(); // RUBY SI

    Cantante uncantante = new Cantante("juanillo");
    System.out.println(discografia.size());

    int j = uncantante.discografia.size(); // RUBY NO
}
```

▼ EN RUBY DIARIO PRIVADO:

```
class Persona
  def escribirDiario
    @diario="Querido Diario, hoy me he traído a una palla"
  end

  def leerDiario
    @puts diario
  end

  def cotillearDiario(compañeroPiso)
    puts compañeroPiso.diario #ESTO EN RUBY NO SE PUEDE pq el diario de otro ES PRIVADO
  end
end
```

```
zoraida=Persona.new
zoraida.escribirDiario("wokfe9fjof")
zoraida.cotillearDiario(mengario) #No puede hacerlo porque no es del obj actual
```

EN RUBY Y EN JAVA EN METODOS HAY DIFERENCIA

Basicamente en ruby no se puede acceder al diario de otro objeto que no sea en el que estamos actualmente, en Java si se puede

EN JAVA DIARIO PRIVADO:

```
class Persona
private String diario;
void escribirDiario(string texto){
    diario=texto;
}
```

23/11/2023

CLASE PERSONA < Granadino

```
class Persona
  def initialize(n)
    @nombre=n
  end
  def saludar
    puts "hola"
  end
  def sorprenderse
    puts "que sorpresa"
  end

  def despedirse
    puts "adiós"
  end
  #Mejor implementacion :
  def despedirse
    return "¡ADIÓS PAYASO!"
  end
end
```



```

class Granadino < Persona

  def initialize(n)
    super(n) #EL SUPER SIRVE PARA INVOCAR AL MISMO METODO EN SU PAE PASANDOLE n
  end

  def sorprenderse
    x=super
    x+="como un gitano"
  end

end

```

Ruby, no existe el tipo estatico,es dinamico, se convierte en lo que quiera, por lo tanto esto si se puede hacer, c sería un variable polimorfica;

c = Granadino.new ("Cecilio")

c = Persona.new ("Fulano")

Java , en java sí existe el tipo estático, por lo tanto esto no se puede hacer, en java se puede hacer polimorfismo pero solo en herencia cuando se trata de subclases

Granadino c = new Granadino ("Cecilio")

c = new Persona ("Fulano")

Una variable en ruby que va cambiando su tipo dinámicamente, es una variable polimorfica

Un Granadino puede ser un maracenero

pero a un maracenero no se le puede añadir un granadino porque le faltan atributos

Maracenero m= new Maracenero("Mengano")

Granadino g=Granadino("Fulano")

g=m //Si es correcto

m=g //Es erroneo porque al padre le faltan atributos para ser maracenero

Granadino g2=new Maracenero("Fulanito")



**YA TIENES UN TÍTULO,
AHORA DA EL SALTO AL
MUNDO LABORAL.**

**POTENCIA TU PERFIL
CON LAS TECNOLOGÍAS
MÁS DEMANDADAS.**



Servicio de carreras
para que encuentres
curro en 180 días.

`m=g2 //Dará error`

Forma para que no dé error:

`m=(Maracenero) g2; // Si hay lo que tu le has dicho sí lo cogerá sin fallos`

//si en el tiempo de ejecución comprobará si es cierto que g2 es un maracenero

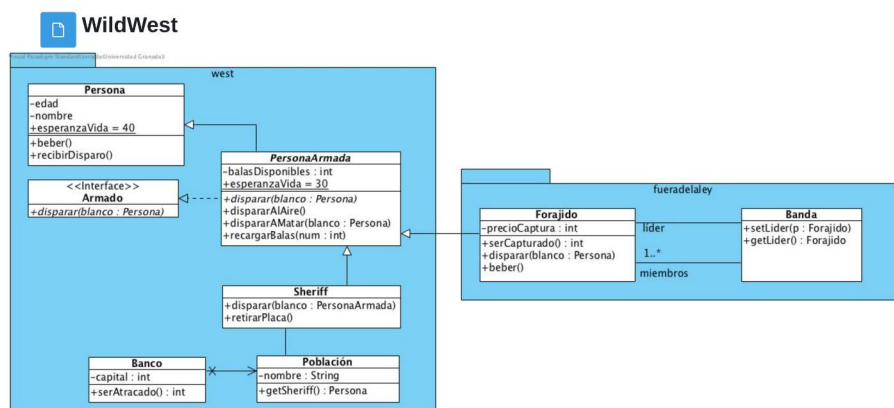
CLASES ABSTRACTAS

Solo cabecera y no codigo, si hay un metodo sin definir

INTERFAZ/INTERFACE SOLO EN JAVA

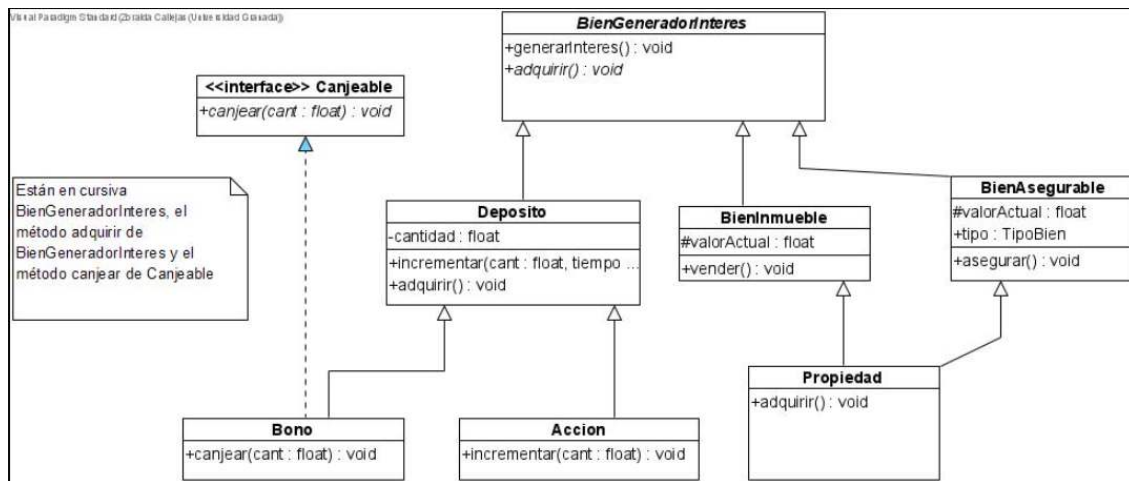
Todos los metodos sin implementar, indican comportamiento

las subclases que hereden de la interfaz deberán implementar estos metodos



```
Sheriff s =new Shellrrif(balas,edad,nombre);
//Constuctor
Sheriff(int balas, int edad, string nombre, Poblacion ogijares){
    super(balas,edad,nombre);
    poblacion = ogijares;
}
//Constructor
PersonaArmada(int balas,int edad, string nombre){
    super(edad,nombre)
    balasDisponibles = balas;
}
```

30/11/2023



EN RUBY NO HAY INTERFACES

Este diagrama para JAVA no se puede hacer ya que no existe la HERENCIA

MULTIPLE

Propiedad por ejemplo hereda de 2 sitios, no se puede, pero vamo a hacer ejemplos importantes :

```
public abstract class BienGeneradorInteres{
    public void generarInteres(){
        System.out.println("genero interes");
    }

    public abstract void adquirir()[] //EN CURSIVA POR LO TANTO INTERFAZ
                                     //DEBERA DE SER IMPLEMENTADO EN HIJOS/NIETOS
}

-----
public class BienInmueble extends BienGeneradorInteres(){
    //Si lo dejas vacio te sale error en la clase, ya que te pide que completes
    //la intefaz del padre, o bien puedes dejarla vacia y que esta clase sea
    //abstracta tambien, pero sus hijos tienen que implementarla, es como una deuda
    //su hijo propiedad deberá de implementar metodo ya, ya que esta no tiene + hijos

    //En el diagrama entonces está mal, la clase BienInmueble tambien seria abstracta

    //Clase abstracta -> Al menos un metodo abstracto aunque sea heredada
    // y no salga en el codigo
}

-----
public class Deposito extends BienGeneradorInteres(){
```

```

    public void incrementar(float cant , int tiempo){
        System.out.println("Incremento deposito" + cant +"durante" + tiempo);
    }

    @Override    //Override porque es redefinir un metodo padre
    public void adquirir(){
        System.out.println("Compro deposito");
    }
}

-----

public class Propiedad extends BienInmueble(){
    @Override
    public void adquirir(){
        System.out.println("Compro Propiedad");
    }
}

-----

public interface Canjeable(){
    public void canjear(float cant);
}

//PRIMERO SUPERCLASE Y LUEGO INTERFAZ (implements)
public class Bono extends Deposito implements Canjeable(){

    @Override //De la interfaz canjeable
    public void canjear(float cant){
        system.out.println("Canjeo " + cant);
    }

}

-----

public class Accion extends Deposito(){
//No pide Override ya que como tiene diferentes atributos, lo considera nuevo metodo
    public void incrementar(float cant){
        System.out.println("Incremento accion en "+ cant);
    }
}

-----

//CREAMOS MAIN DE PRUEBA
public class Main(){
    public static main (String [] args){
        Accion b = new Accion();
//Llamadas a funciones de prueba y explicasion;
        //Heredado de BienGeneradorInteres
        b.generarInteres();

        //Heredado de deposito
        b.incrementar(20f,10); //Llama a metodosdiferentes por atributos que tienen
        b.adquirir();

        //Propio
        b.incrementar(20f);
    }
}

```



**YA TIENES UN TÍTULO,
AHORA DA EL SALTO AL
MUNDO LABORAL.**

**POTENCIA TU PERFIL
CON LAS TECNOLOGÍAS
MÁS DEMANDADAS.**



Servicio de carreras
para que encuentres
curro en 180 días.

PONTE A PRUEBA DE PRADO → RECOMENDADO → TAMBIEN TRABAJO
AUTÓNOMO



PIDE MÁS INFO